

parameters. The observed outputs are shown below:

```
$ curl localhost:3000/user
<h1>500 Internal Server Error</h1>Param: name not found!

$ curl localhost:3000/user?name=Shakthi
<h1>Hello Shakthi</h1>
```

The *Hspec* testing framework can be used for integration testing of the Web application. Install the required dependencies as shown below:

```
$ cabal install happy hspec hspec-wai hspec-wai-json
```

The content type example has been updated to use *Hspec*, as illustrated below:

```
-- content-type-spec.hs

{-# LANGUAGE OverloadedStrings, QuasiQuotes #-}
module Main (main) where

import Data.Monoid
import Data.Text

import           Network.Wai (Application)
import qualified Web.Scotty as W
import           Data.Aeson (object, (.=))

import           Test.Hspec
import           Test.Hspec.Wai
import           Test.Hspec.Wai.JSON

main :: IO ()
main = hspec spec

app :: IO Application
app = W.scottyApp $ do
  W.get "/hello.txt" $ do
    W.text "Hello, World!"

  W.get "/hello" $ do
    W.html $ mconcat ["<h1>", "Hello, World!", "</h1>"]

  W.get "/hello.json" $ do
    W.json $ object ["text" .= ("Hello, World!" :: Text)]

spec :: Spec
spec = with app $ do
  describe "GET /" $ do
    it "responds with text" $ do
      get "/hello.txt" `shouldRespondWith` "Hello, World!"

    it "responds with HTML" $ do
```

```
get "/hello" `shouldRespondWith` "<h1>Hello, World!</h1>"

    it "responds with JSON" $ do
      get "/hello.json" `shouldRespondWith` [json|{text:
"Hello, World!"}|]
```

You can compile the above code as follows:

```
$ ghc --make content-type-spec.hs
Linking content-type-spec ...
```

The following output is observed when you run the above built test executable:

```
$ ./content-type-spec

GET /
  responds with text
  responds with HTML
  responds with JSON

Finished in 0.0010 seconds
3 examples, 0 failures
```

Please refer to the *hspec-wai* Web page at <https://github.com/hspec/hspec-wai> for more information.

Template support is available through many Haskell packages. The use of the *blaze-html* package is demonstrated below. Install the package first using the following command:

```
$ cabal install blaze-html
```

Consider a simple Web page with a header and three unordered lists. Using *blaze-html*, the template can be written in Haskell DSL as follows:

```
-- template.hs

{-# LANGUAGE OverloadedStrings #-}

import Web.Scotty as W
import Text.Blaze.Html5
import Text.Blaze.Html.Renderer.Text

main :: IO ()
main = scotty 3000 $ do
  get "/" $ do
    W.html . renderHtml $ do
      h1 "Haskell list"
      ul $ do
        li "http://haskell.org"
        li "http://learnyouahaskell.com/"
        li "http://book.realworldhaskell.org/"
```